WASP course on Software Engineering

Marco Antonio Corallo

September 2026

1 Introduction

As a PhD student in cybersecurity, my research interests span various domains within security and privacy. My current focus is on Machine Learning (ML) security, specifically about the security of Neuro-Symbolic (NeSy) models. NeSy models offer a principled approach to deploying more transparent, efficient, and reliable AI systems [3, 7] by integrating a logical reasoning layer into standard ML architectures, particularly Neural Networks (NNs). My research primarily investigates whether and how this reasoning layer can be leveraged to enhance overall model robustness and ultimately serve as a protective barrier for standard NNs. On the other hand, I'm interested in understanding how NeSy models can be used to enforce soundness and security in safe-critical ML-based tools, as well as to build more reliable tools for security tasks.

So far, I have investigated the robustness of NeSy models against backdoor attacks [1], demonstrating that this paradigm yields models that are inherently more robust than baseline neural networks. However, backdoor attacks remain a real threat, as attackers can still tailor the data poisoning process by selecting vulnerable target labels [4]. My ongoing and future work will explore techniques to enhance NN robustness through the addition of a reasoning layer, alongside evaluating the application of NeSy models on security tasks.

2 Lecture principles

The topics I chose came from the second lecture, namely *"Quality Assurance and Testing in SE, QA for ML, ML for QA"*. The lecture introduces the concepts of testing, validation and verification, along with the main techniques and differences. Among these techniques, intrinsically inherent to security, at least two fit to NeSy models.

2.1 Static techniques: a static analyzer for NeSy models' robustness

The first idea consists in the design of a static analyzer for NeSy models, to verify properties of the complete reasoning pipeline, rather than analyzing the neural and symbolic components independently.

Existing works for NNs verification [6] consider a set of possible inputs, for example all images within a bounded perturbation of a reference image, to compute then sound bounds on the output of each neural predicate.

In most of the NeSy models, these outputs correspond to probabilities assigned to symbolic facts. Instead of returning a single probability for each fact, the analyzer produces an interval containing every probability that the neural network can generate for the considered input set.

The analyzer would first ground the relevant part of the logic program for a given query and compiles it into the same type of arithmetic circuit used during probabilistic inference. It then evaluates this circuit over probability intervals. Conjunctions, disjunctions, and probabilistic choices are replaced by sound abstract operations that propagate lower and upper probability bounds. The final result would therefore be an interval for the probability of the query. A property can be certified when this interval is sufficient to establish it. For example, the analyzer may prove that the probability of the correct answer is always greater than a required threshold, or that no competing answer can become more probable under the allowed input perturbations.

The final goal would be the deploy of an analyzer able to certify robustness properties that cannot be obtained by inspecting neural predictions alone. In particular, symbolic rules may either amplify local uncertainty or prevent it from affecting the final query, as well as potential backdoor effects [4].

2.2 Design-By-Contract on NNs through the logic layer

This second idea aims to introduce Design-by-Contract for NNs by using the logical layer of a NeSy model to define pre-conditions, post-conditions and invariants.

In this setting, the *contract* is represented by a set of logic rules associated with the neural predicate. Pre-conditions describe the inputs for which the neural component is expected to operate. For example, they may require that an input belongs to a particular domain, that some contextual information is available, or that two input objects satisfy a known relation. Post-conditions constrain the output of the neural predicate, possibly in relation to symbolic background knowledge. Finally, invariants express properties that should remain valid across different predictions, such as mutual exclusion between classes, consistency between related neural predicates, or compatibility with domain rules.

This idea would bring an effective design strategy of software engineering to NNs, constraining the execution to safe states and treating them as less opaque and more reliable components.

3 Guest-lecture principles

The picked topics came from the lecture on *Requirements Engineering*, held by Dr. Frattini.

3.1 NeSy as the next step of formal RE + LLMs

This idea builds on top of the topic "Formal requirements engineering and large language models" [5]. In particular, on top of the idea of using formal methods to improve the correctness and trustworthiness of LLM-based requirements engineering. The nature of NeSy models fit well as the natural next step of this mix, since it combines learned language processing and explicit symbolic reasoning within a single system.

As a first idea, I imagine the following workflow: the neural component processes requirements expressed by stakeholders in natural language. It identifies relevant entities, system functions, actors, assets, constraints, and relations between requirements. Instead of directly producing a final formal specification, the neural component maps this information to probabilistic symbolic facts. For example, it may state with a given confidence that a sentence describes an access-control requirement, that a database is a protected asset, or that a user action requires authentication. These facts are then evaluated by a logic program containing domain knowledge and requirements-engineering rules.

The symbolic layer performs tasks for which explicit reasoning is important. It can check whether requirements are mutually consistent, detect missing information, derive additional requirements, and maintain traceability between stakeholder statements and formalized properties. A rule may identify

that an operation on sensitive data requires authorization, or that a requirement is incomplete because it defines a protected resource without identifying the actors permitted to access it.

3.2 Stakeholder Elicitation (through NeSy)

While this second topic is not really aligned with the security of NeSy models, I found it interesting as an application of NeSy models in general. The idea is to develop a neuro-symbolic approach to support the two main related tasks: identifying the stakeholders and making the relationships between them explicit.

In my idea, the neural component processes heterogeneous natural-language sources and extracts candidate stakeholders. Instead of making a final decision, the neural model produces probabilistic facts describing each candidate and its possible role. For example, it may assign different confidence values to the hypotheses that an organization belongs to some stakeholder class or another one. This uncertainty is preserved and passed to the symbolic component. The logic layer contains general rules about stakeholder roles and domain-specific knowledge. These rules can infer stakeholders that are not explicitly named in the available documents.

The second part of the approach constructs a stakeholder relationship graph. Neural predicates extract possible relations from text, while logical rules normalize and complete them. Relations may express communication, responsibility, dependency, data exchange, trust, conflict, or influence, possibly deriving new relations from others. Each relation is linked to the evidence and rules that produced it, allowing the requirements engineer to inspect and correct the resulting graph.

4 Data scientists, software engineers, and AI engineers

What differences do the chapters describe between data scientists and software engineers? Do you agree with this distinction? Why or why not? Chapter 1 describes data scientists as typically trained in statistics and ML, working experimentally with data and models, and emphasizing predictive performance. Software engineers focus on requirements, architecture, testing, deployment, maintenance, and trade-offs such as cost and reliability. The authors explicitly call this distinction an oversimplification, but it helps to give an idea of the two roles and the differences between them. I do agree with this distinction, because the roles, main tasks and objectives require completely different types of background and attitude. That doesn't mean the first person cannot engineer or the second cannot experiment, but each of them has a different expertise and attitude.

The chapters argue that production ML requires a system-wide view, not only a model-centric view. How does this apply to your own research area? A system-wide view is useful for neuro-symbolic security because it starts from the security objective and considers how all the components, users, and environment contribute to it. Combining a neural model with a symbolic reasoner is only part of that picture. We also need to consider how evidence is collected, how an alert reaches an analyst, and what action follows. This helps us choose where neural predictions, symbolic checks, and human judgment can contribute most effectively. It also works in both directions: operational needs shape the requirements of the neuro-symbolic components, while their capabilities shape which responses we can reasonably automate.

Do you think data scientists and software engineers will remain distinct roles, or will both need to learn more of each other's skills? Explain. I expect the roles to remain recognizable while sharing more skills. Data scientists should understand versioning, integration tests, deployment constraints, and

monitoring. Software engineers should understand leakage, distribution shift, uncertainty, and sound evaluation. That doesn't require everyone to become equally good at everything. For this security example, a modeling specialist and a systems specialist could jointly define how uncertain observations enter the rule engine. Shared responsibility for that interface seems more realistic than handing over a model and hoping another team makes it work.

Where does "AI Engineering" fit in this picture: is it mostly software engineering for AI systems, data science made more systematic, MLOps/model engineering, or something partly distinct? Chapter 2 notes that terminology is inconsistent: AI engineering often refers to pipelines and MLOps, although broader uses exist. I would use the broader meaning: engineering the complete AI-enabled system. Its foundation is software and systems engineering; systematic data science supplies experimental evidence, and MLOps supplies repeatable model delivery and operation. It is also partly distinct in its emphasis on managing behavior learned from data or elicited through prompts. Calling all of this MLOps would understate the work on requirements, interactions, and consequences.

How do foundation models, AI coding assistants, RAG systems, prompt pipelines, and agentic workflows change the boundaries between these roles? With a foundation model, a developer can build an AI feature without training its underlying model. I'd expect more effort to move toward selecting models, preparing context, and evaluating behavior. In RAG, retrieval quality, document freshness, and access permissions become part of answer quality. Prompt pipelines need versioning and tests across stages. Agentic workflows add decisions about tools, stopping conditions, and recovery, making the architecture consequential. On the other hand, AI coding assistants help developers to implement software and prototypes, but generated code still needs review and testing, perhaps more than human-written code.

5 Paper analysis

I picked both the papers from CAIN 2026, among the ones with available pre-print.

5.1 How to Build AI Agents by Augmenting LLMs with Codified Human Expert Domain Knowledge? A Software Engineering Framework [8]

Core ideas and their SE importance The paper turns expert know-how into reusable agent components: interview experts, encode their knowledge, test, and iterate. In its Siemens simulation visualization case, procedural rules become Python functions, while contextual design guidance goes into prompts. A classifier routes requests, and RAG supplies software-specific documentation and code examples. On the software engineering side, the lesson is that useful AI needs domain knowledge built into the surrounding software to reduce repeated expert support and to give developers concrete rules to inspect and test.

Relation to your research I read this combination of learned language capabilities and explicit rules as a practical neuro-symbolic pattern. However, the rule reports feed the LLM's prompt, without enforcing its output. The paper also excludes code security from evaluation. Would be very interesting to me to work on an extension to make security requirements executable outside the LLM: generated code may read the selected dataset, but cannot overwrite simulation inputs or access another project's files. The application would enforce these permissions regardless of the prompt. I would also check whether misleading

retrieved examples can persuade the agent to skip required analysis steps. The research question is whether the neural component can work flexibly while the symbolic checks remain dependable under hostile inputs.

Integration into a larger AI-intensive project The scenario I think is about an AI-powered IT helpdesk that manages support tickets and suggests solutions to employees. Someone could have login problems for which asks to helpdesk. Following the paper's approach, the assistant would retrieve internal troubleshooting guides and use rules collected from experienced support staff to suggest the next steps. This would help junior staff handle common problems without constantly asking senior colleagues.

I see a natural integration of neuro-symbolic security here: the LLM interprets the request and explains the solution, while explicit rules control which actions are allowed. For example, the application for helpdesk would prevent a password reset until the employee's identity has been verified. This is an idea of extension of the paper's framework, combining reusable expert knowledge with security checks enforced outside the LLM.

Adaptation of your research Focusing on NeSy models for security specifically, the problem I see is that a correct expert rule can still lead to an unsafe decision if the neural model supplies a false fact. This is pretty common on LLMs under prompt injection attacks, where an user can mislead the model's prediction. I would investigate a neuro-symbolic approach where the neural component extracts claims from requests, and the symbolic component reasons about those claims together with their source. A rule requiring approval could only be satisfied by evidence from the company's approval system. Missing evidence would trigger a request for verification. This would also extend the paper's approach to embedding expert knowledge by studying whether those rules remain dependable when they receive imperfect neural interpretations.

5.2 SETA: Statistical Fault Attribution for Compound AI Systems [2]

Core ideas and their SE importance SETA addresses a practical debugging problem: when several AI models work together, a bad final output does not tell us which component needs fixing. It combines metamorphic testing with execution traces. Engineers define expected relations between outputs on original and perturbed inputs, combine these checks per component, and record output and routing changes. Aggregating violations produces normalized scores that help prioritize suspicious modules without requiring labels for every test.

Relation to your research For neuro-symbolic security, I see a useful connection. I would treat the boundary between neural predictions and symbolic facts as a key testing point. A manipulated image might produce a wrong signal label that a rule engine accepts, leading to an unsafe decision even when the rules execute correctly. Explicit contracts could check whether the necessary evidence exists before a rule fires, while traces could show which prediction supplied that evidence.

Integration into a larger AI-intensive project I imagine an invoice-processing feature inside a company's accounting platform. An OCR model reads scanned invoices, and another model extracts the supplier, items, and total amount. The results appear in a dashboard for staff to review. Developers could use SETA to test the pipeline with slightly blurred or rotated scans that remain readable. By checking intermediate outputs and collecting fault scores, they could investigate whether problems arise in text recognition or field extraction. A simple neuro-symbolic extension would combine these neural models

with explicit rules: the total should match the listed amounts, and payment details should match an approved supplier record. Violations would trigger human review. This could help catch some incorrect or manipulated inputs, while SETA would help developers investigate failures after model updates.

Adaptation of your research On my topic, would be interesting to investigate which simple symbolic checks prevent neural errors from becoming security problems. For the invoice example, I would compare the same AI pipeline with and without rules that check totals and approved supplier details. I would test normal, corrupted, and deliberately manipulated invoices. SETA-like's traces and fault scores could help identify components worth investigating, while I would separately measure how many incorrect invoices get accepted and how many valid ones get unnecessarily flagged.

6 Evidence, originality, and GenAI-use transparency

All the sources used and cited below are peer-reviewed papers and, in most cases, representative and highly cited works. For these reasons, they are trusted sources for the scientific community, as well as for me.
The papers cited in Section 5 refers to pre-print, but they have been selected from CAIN 2026 as suggested during the lecture.
I have used ChatGPT to rephrase some sentences and/or sections, while the ideas, analyses, and responses are my own, and I take responsibility for them.

References

[1] Mauro Barni. "Backdooring Deep Learning Architectures: Threats and (some) Opportunities". In: *Proceedings of the 6th International Conference on Information Systems Security and Privacy, ICISSP 2020, Valletta, Malta, February 25-27, 2020*. Ed. by Steven Furnell et al. SCITEPRESS, 2020, pp. 15–16.

[2] Sayak Chowdhury and Meenakshi D'Souza. "SETA: Statistical Fault Attribution for Compound AI Systems". In: *arXiv preprint arXiv:2601.19337* (2026).

[3] Giovanni Ciatto et al. "Symbolic Knowledge Extraction and Injection with Sub-symbolic Predictors: A Systematic Literature Review". In: *ACM Comput. Surv.* 56.6 (2024), 161:1–161:35. DOI: 10.1145/3645103. URL: https://doi.org/10.1145/3645103.

[4] Marco Antonio Corallo et al. *Does Reasoning Mitigate Backdoor Attacks? A Neuro-Symbolic Perspective*. 2026. arXiv: 2609.00464 [cs.CR]. URL: https://arxiv.org/abs/2609.00464.

[5] Alessio Ferrari and Paola Spoletini. "Formal requirements engineering and large language models: A two-way roadmap". In: *Inf. Softw. Technol.* 181 (2025), p. 107697. DOI: 10.1016/J.INFSOF.2025.107697. URL: https://doi.org/10.1016/j.infsof.2025.107697.

[6] Timon Gehr et al. "Ai 2: Safety and robustness certification of neural networks with abstract interpretation". In: *Security and Privacy (SP), 2018 IEEE Symposium on*. 2018.

[7] Giuseppe Marra et al. "From statistical relational to neurosymbolic artificial intelligence: A survey". In: *Artificial intelligence* 328 (2024), p. 104062.

[8] Choro Ulan uulu et al. *How to Build AI Agents by Augmenting LLMs with Codified Human Expert Domain Knowledge? A Software Engineering Framework*. 2026. arXiv: 2601.15153 [cs.AI]. URL: https://arxiv.org/abs/2601.15153.